

Theoretical Computer Science

Probabilistic Algorithms

Technische Hochschule Rosenheim
Sommer 2026
Prof. Dr. Jochen Schmidt

- Pseudo random numbers
- Monte-Carlo Methods
 - example: probabilistic primality tests

- either an **approximation** of the actual result
 - typically by random sampling of the value range, using many sampling points
 - e.g.: (numerical) calculation of integrals, computer graphics (Photon Mapping)
- or a result that is only **correct with a certain probability**
 - e.g., primality tests:
 - Result: Number is not prime $\rightarrow$ always correct
 - Result: Number is prime $\rightarrow$ only correct with a very high probability
- in many cases, this is the only way to make the calculation practicable
- random numbers are required

- Calculate approximation of the value of

$$F = \int_a^b f(x) dx$$

- Idea:

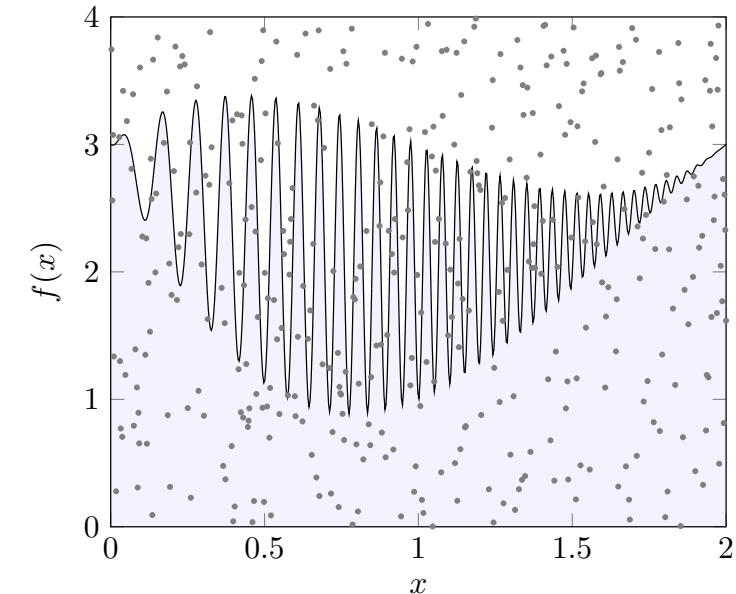
- determine bounding box (area R) of $f(x)$ – given by maxima/minima in the interval $[a; b]$
- generate N pairs of random numbers that define coordinates (x, y) within the rectangle
- Count how many points N_f are below the function $f(x)$
- An approximate value of F is given by

$$F = R \frac{N_f}{N}$$

- Works the same way for multi-dimensional functions

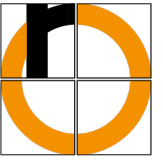
- Example: Calculate integral

$$F = \int_0^2 \left(2 + (x-1)^2 + \sin[40 \cdot (x+x^2)] \cdot x \cdot (x-2)^2 \right) dx$$

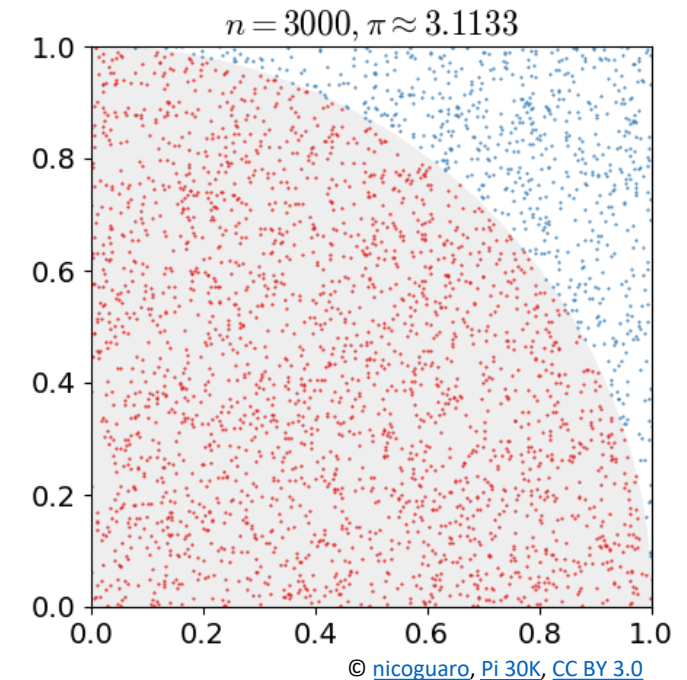


- Monte-Carlo with 10,000 random points:
 $F = 4.671$
- Exact value (to three decimal places):
 $F = 4.667$

Approximation of π



- Circle area: $A = r^2\pi \rightarrow \pi = A / r^2$
- Use quarter circle as a function
- Test whether $\sqrt{x^2 + y^2} \leq r$ holds for random points (x, y)
 - then the point is in the circle
 - Calculate area as before in the integral example
- We obtain $\pi/4$ from the equation above



- **True** random numbers
 - use natural random processes
 - radioactive decay, thermal noise of electronic components, quantum physical processes
 - cannot be generated by standard computers
- **Pseudo** random numbers (*Pseudozufallszahlen*)
 - algorithmic calculation of “random numbers”
 - typically:
 - iterative calculation
 - sequence is repeated after a certain amount of numbers
 - deterministic: using the same initial value, the sequence is exactly reproducible
 - Initialization, e.g., using
 - Current system time
 - Current state of memory, registers, location of r/w head of hard drives, ...

- Important probability distributions:
 - uniform distribution (*Gleichverteilung*)
 - Normal distribution (Gaussian distribution, *Normalverteilung/Gaußverteilung*)
- Basic (pseudo) random number generators generate **uniformly distributed** numbers
 - which can be used to derive normally distributed ones
- Are the numbers “good”? Do they correspond to the desired distribution?
→ use statistical tests, e.g.,
 - χ^2 -Test (Pearson, 1900),
 - Kolmogorov-Smirnov-Test (1933/39).

- widespread: **Linear Congruential Generator** (LCG, *linearer Kongruenzgenerator*)
 - Lehmer 1949

- Calculate (integer) random numbers using the recurrence $x_{n+1} = (a x_n + c) \bmod m$

- where

- m modulus $0 < m$
- a multiplier $0 \leq a < m$
- c increment $0 \leq c < m$
- x_0 initial value (**seed**) $0 \leq x_0 < m$

- generates integers in the interval $[0; m - 1]$

- How do we choose the parameters?

Example: binary random sequence with $m = 2$?

$$x_{n+1} = (a x_n + c) \bmod m$$

- $a = 0$ and $c = 0/1$:
 $x_{n+1} = 0 \bmod 2$ or $x_{n+1} = 1 \bmod 2$
 - 0, 0, 0, 0, 0, ... $\rightarrow$ not very random
 - 1, 1, 1, 1, 1, ... $\rightarrow$ not very random
- $a = 1$ and $c = 0$: $x_{n+1} = x_n \bmod 2$
 - always returns the initial value x_0
- $a = 1, c = 1, x_0 = 0$: $x_{n+1} = (x_n + 1) \bmod 2$
 - 0, 1, 0, 1, 0, 1, 0, 1, 0, ... $\rightarrow$ not very random
- $a = 1, c = 1, x_0 = 1$: $x_{n+1} = (x_n + 1) \bmod 2$
 - 1, 0, 1, 0, 1, 0, 1, 0, 1, ... $\rightarrow$ not very random

Two other examples

- $m = \text{arbitrary}, c = 1, a = 1, x_0 = 0$:

$$x_{n+1} = (x_n + 1) \bmod m$$

- 0, 1, 2, 3, ..., $m - 1$, 0, 1, 2, 3, ...
 $\rightarrow$ not very random

- $m = 10, c = 7, a = 7, x_0 = 7$:

$$x_{n+1} = (7x_n + 7) \bmod 10$$

- 7, 6, 9, 0, 7, 6, 9, 0, ...
 $\rightarrow$ not very random

- Conclusion: Choice of parameters is extremely important

$$x_{n+1} = (a x_n + c) \bmod m$$

- $a \geq 2$
 $a = 0$ and $a = 1$ do not generate a random sequence
(this obviously implies $m \geq 2$)
- $c = 0$ has the effect of
 - faster calculation
 - shorter period length
- m : as desired
 - in practice: based on word length of CPU, e.g., 2^{32}
- We get **maximum period length** m if and exactly if
 - c and m have no common prime factors: $\gcd(c, m) = 1$
 - for each prime factor p of m : $a - 1$ is a (integer) multiple of p
 - if m is a multiple of 4: $a - 1$ is also a multiple of 4

Examples from actual implementations

- `stdlib` in `gcc`
 - $m = 2^{32}$
 - $a = 1103515245$
 - $c = 12345$
- Numerical Recipes
 - $m = 2^{32}$
 - $a = 1664525$
 - $c = 1013904223$
- Java `Random` Class
 - $m = 2^{48}$
 - $a = 25214903917$
 - $c = 11$
 - only (upper) 32 bits of the result are used
 - upper bits produce longer periods

Given: uniformly distributed **integer** random number r in the interval $[0; m - 1]$

- Conversion to another **integer interval** $[A, B]$:
$$x = A + (r \bmod (B - A + 1))$$
- Conversion to another **real-valued interval** $[A, B]$:
$$x = A + r (B - A) / (m - 1)$$
- Conversion to **normal distribution**, in two steps:
 - Conversion to standard normal distribution $\mu = 0, \sigma = 1$
 - Polar method by Box, Muller, Marsaglia (1958/1962)
 - Conversion to any other normal distribution
 - let x be normally distributed with $\mu = 0, \sigma = 1$
 - then $ax + b$ is normally distributed with $\mu = b, \sigma = a$

- Generate two uniformly distributed random numbers v_1 and v_2 in interval $[-1; +1]$
- Calculate $S = v_1^2 + v_2^2$
 - repeat these steps until $S < 1$
 - this is necessary on average 1.27 times, with standard deviation 0.587
- This results in two standard normally distributed random numbers x_1 and x_2 as follows:

$$x_1 = v_1 \sqrt{\frac{-2 \ln S}{S}} \quad x_2 = v_2 \sqrt{\frac{-2 \ln S}{S}}$$

- Given: Natural number n
- Question: Is n a prime number?
- Application: public key cryptography
 - e.g., RSA (1978)
 - public key: product of two very large prime numbers
 - 1024 – 4096 binary digits for key (= approx. 308 to 1233 decimal digits)
- Methods for primality tests
 - Sieve of Eratosthenes – exponential runtime
 - AKS-Test (2002) – polynomial runtime $\rightarrow \text{PRIMES} \in \text{P}$
 - too slow for practical purposes
 - Instead: probabilistic primality tests

- Pierre de Fermat (ca. 1607 – 1665)
- If p is prime, then for any natural number a that is not a multiple of p (just use $a < p$)
$$a^{p-1} \bmod p = 1$$
- The reverse is not true
 - there are numbers p that satisfy the equation even though they are not prime
 - e.g., $p = 11 \cdot 31 = 341 \quad \rightarrow \quad 2^{340} \bmod 341 = 1$
- **Fermat primality test**
 - check for many (randomly generated) a whether Fermat's little theorem is satisfied
 - if there is an a for which it fails: p is **definitely** not prime
 - otherwise: inconclusive (interpreted as: p is **probably** prime)
 - Problem: there are numbers p
 - that are not prime,
 - but for which Fermat's little theorem is satisfied for **any** $a \rightarrow$ Carmichael numbers

- published in 1976
- any odd number n can be represented as $n = 1 + q2^k$ (with q an odd number)
- if n is prime then according to Fermat: $a^{n-1} \bmod n = 1 \rightarrow a^{q2^k} \bmod n = 1$
- in addition, the following applies:
 $a^q \bmod n = 1$ or $a^{q2^r} \bmod n = n - 1 \equiv -1$ for some r with $0 \leq r \leq k - 1$
- **Idea:** Calculate the sequence $(a^q, a^{2q}, a^{4q}, \dots, a^{q2^{k-1}}, a^{q2^k})$
 - use as many random “ a ” from the interval $[2; n - 1]$ as required
- If n is a prime number, the sequence must have one of the following forms:
 - $(1, 1, 1, \dots, 1)$
 - $(x_1, x_2, x_3, \dots, x_m, -1, 1, 1, \dots, 1)$ x_i arbitrary numbers, m may be zero

- Test $n = 11$ with $a = 2$

$$11 = 1 + 5 \cdot 2 \rightarrow q = 5, k = 1$$

$$2^5 \bmod 11 = 10 \equiv -1$$

$$2^{10} \bmod 11 = 1$$

→ probably prime,
maybe test some other a

- Test $n = 65$ with $a = 2$

$$65 = 1 + 1 \cdot 2^6 \rightarrow q = 1, k = 6$$

$$2^1 \bmod 65 = 2$$

$$2^2 \bmod 65 = 4$$

$$2^4 \bmod 65 = 16$$

$$2^8 \bmod 65 = 61$$

$$2^{16} \bmod 65 = 16$$

$$2^{32} \bmod 65 = 61$$

$$2^{64} \bmod 65 = 16$$

→ definitely not prime

- Test $n = 561$ with $a = 2$

$$561 = 1 + 35 \cdot 2^4 \rightarrow q = 35, k = 4$$

$$2^{35} \bmod 561 = 263$$

$$2^{70} \bmod 561 = 166$$

$$2^{140} \bmod 561 = 67$$

$$2^{280} \bmod 561 = 1$$

$$2^{560} \bmod 561 = 1$$

→ definitely not prime

561 is the smallest Carmichael number

- Note: The calculation is greatly simplified if you use the following relationship:

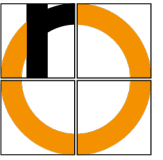
$$(x \cdot y) \bmod n = ((x \bmod n) \cdot (y \bmod n)) \bmod n$$

- The most widely used primality test
- Error probability
 - the result “not prime” is always 100% correct
 - the result “prime” is for a single randomly chosen a is incorrect with a probability of $\frac{1}{4} = 25\%$
 - by repeated testing with different values for “ a ” we can make it arbitrarily small
 - e.g.: test 12 times $\rightarrow$ error probability $(\frac{1}{4})^{12} = 0.00000596\%$
- For compound numbers, the test does not provide any information about the prime factors!
 - Prime factorization is much more difficult
 - how difficult exactly, is currently unknown
 - probably not in P
 - but highly likely not NP-complete
(otherwise, we would get $P = NP$, since factorization has been proven to be both, in NP **and** co-NP)

How do we find a prime number?

- Use random number generator to get an odd number n from desired range
 - for cryptographic applications there are special cryptographically-secure pseudorandom number generators
- Use, e.g., Miller-Rabin test to check whether n is prime
 - this again requires generating a certain number of random integers to test (the “ a ” parameter)
- For 2048-Bit RSA two primes with approx. 308 decimal digits each are required
- How likely is it to find a prime in this range if we just pick some random number?

How Many Prime Numbers $< n$ Are There?



- Let $\pi(n)$ the number of prime numbers less than or equal to n

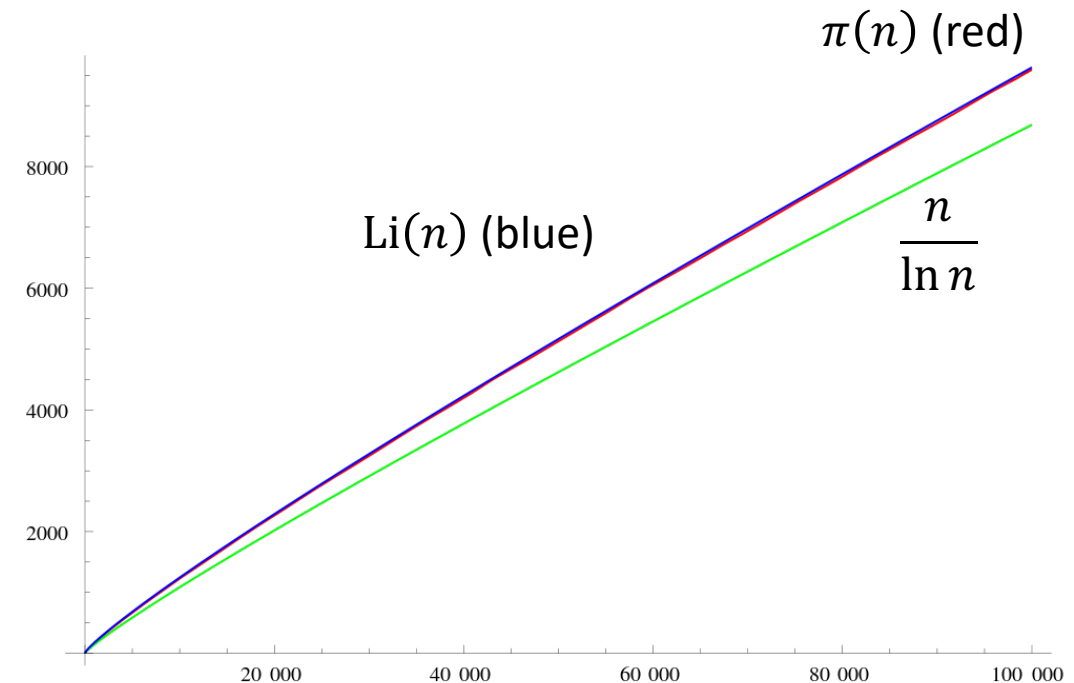
- **Prime number theorem (Primzahlssatz):** $\pi(n) \sim \frac{n}{\ln n}$

- the actual number is even slightly larger
- conjectured by Gauss (1792/93), Legendre (1797/98)

- better approximation: $\pi(n) \sim \text{Li}(n)$
where $\text{Li}(n) = \int_2^n \frac{1}{\ln x} dx$

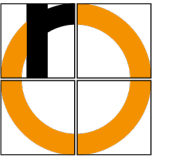
- conjectured by Dirichlet (1838)

- Proofs by: Hadamard and Vallée-Poussin (1896)

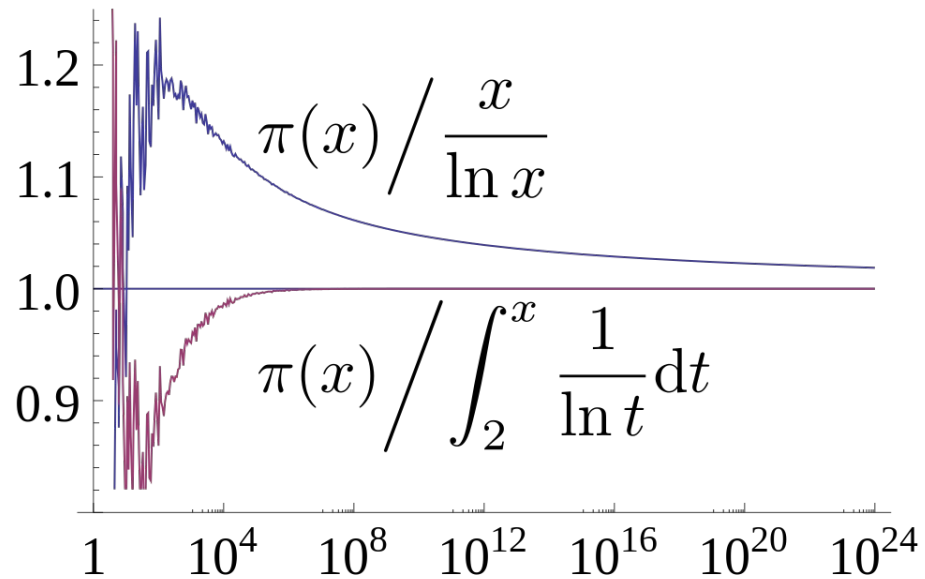


© Noel Bush, [PrimeNumberTheorem](#), CC BY-SA 3.0

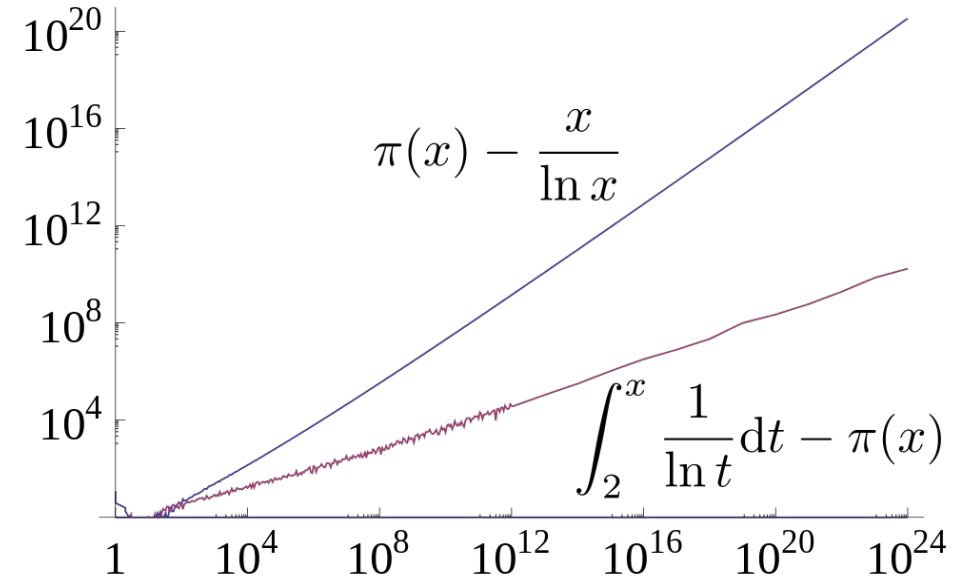
How Many Prime Numbers $< n$ Are There?



convergence



absolute error



- Probability p that a randomly drawn number $x < n$ is prime (using relative frequency):

$$p(x \text{ prime}) = \frac{\frac{n}{\ln n}}{n} = \frac{1}{\ln n}$$

- Probability p that a randomly drawn **odd** number $x < n$ is prime :

$$p(x \text{ prime}) = \frac{\frac{n}{\ln n}}{n/2} = \frac{2}{\ln n}$$

- Probability p , that an odd random number x **from the interval** $[10^a; 10^{a+1}]$ is prime:

$$p(x \text{ prime}) = \frac{\frac{10^{a+1}}{\ln 10^{a+1}} - \frac{10^a}{\ln 10^a}}{1/2 (10^{a+1} - 10^a)}$$

- first-order approximation:

$$\begin{aligned} p(x \text{ prime}) &= \frac{\frac{10^{a+1}}{\ln 10^{a+1}}}{1/2 \cdot 10^{a+1}} = \frac{2}{\ln 10^{a+1}} \\ &= \frac{2}{(a+1)\ln 10} \end{aligned}$$

... which is the same as on the left

Probability p , that an odd random number x from the interval $[10^{300}; 10^{301}]$ is prime:

$$p(x \text{ prime}) = \frac{2}{301 \ln 10} \approx 0.00288567 \quad \text{approx. 0.29\%}$$

Exact value for interval without first-order approximation: 0.00288461 approx. 0.29%

Better approximation by Li-integral:

$$p(x \text{ prime}) = \frac{\text{Li}(10^{a+1}) - \text{Li}(10^a)}{1/2 (10^{a+1} - 10^a)}$$

Result for the example: 0.00321095 approx. 0.32%

Probability that out of 100 numbers drawn, at least one is prime:

$$1 - (1 - 0.0032)^{100} \approx 27.4\%$$

- probabilistic algorithms
 - deliver solutions that are likely to be correct (e.g., primality tests)
 - or approximations of solutions (e.g., numerical integration)
- there are many other applications of the Monte-Carlo method
 - physics simulations
 - micro-electronics
 - finance
 - ...
- in all cases good pseudo-random numbers are required

- H. Ernst, J. Schmidt und G. Beneken: *Grundkurs Informatik*. Springer Vieweg, 8. Aufl., 2023.
- D.E. Knuth: *The Art of Computer Programming*. Vol. 2, *Seminumerical Algorithms*. 3rd edition, Addison-Wesley, 1998.
- H. Scheid: *Zahlentheorie*. 2nd edition, BI Wissenschaftsverlag, 1994.